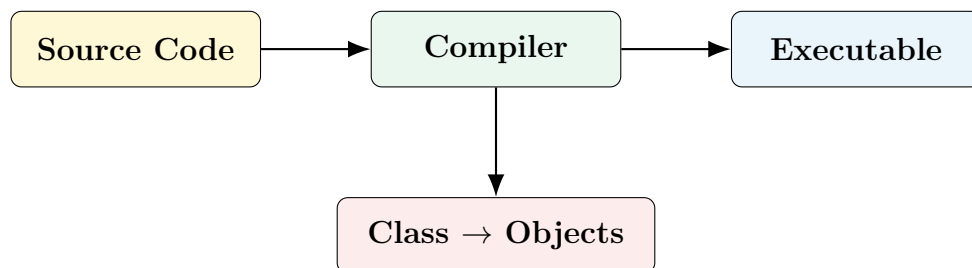


OBJECT ORIENTED PROGRAMMING SYSTEM

Course Code: 25CS301

Lecture 2

Structure of a C++ Program, Classes, Objects and Scope
Resolution



Prepared for B.Tech III Semester

Suggested classroom duration: 60–75 minutes

Based on Unit 1 of the autonomous syllabus

Contents

1	Lecture Overview	5
1.1	Position of this lecture in Unit 1	5
1.2	Learning outcomes	5
2	Recap of Lecture 1	5
3	Anatomy of a C++ Program	6
3.1	Line-by-line explanation	6
3.1.1	The include directive	6
3.1.2	The main function	6
3.1.3	The function body	7
3.1.4	The output statement	7
3.1.5	The return statement	7
4	From Source Code to Executable Program	7
4.1	Preprocessing	8
4.2	Compilation	8
4.3	Assembly	8
4.4	Linking	8
5	Preprocessor Directives and Header Files	8
5.1	Preprocessor directives	8
5.2	Standard headers	9
5.3	Angle brackets and quotation marks	9
6	Namespaces and the Scope Resolution Operator	9
6.1	Using directive	10
6.2	Selective using declaration	10
6.3	User-defined namespace	10
7	Statements, Blocks and Comments	11

7.1	Statements	11
7.2	Blocks	11
7.3	Comments	11
8	Tokens in C++	11
8.1	Keywords	12
8.2	Identifiers	12
8.3	Literals	13
9	From Structure to Object-Oriented Structure	13
10	Class Syntax	13
10.1	First class example	14
10.2	Dry run	15
11	Class versus Object	15
12	State, Behaviour and Identity	15
12.1	State	16
12.2	Behaviour	16
12.3	Identity	16
13	Data Members and Member Functions	16
13.1	Data members	16
13.2	Member functions	16
14	Creating Multiple Objects	17
14.1	Conceptual memory view	18
15	Access Specifiers: A First Look	18
16	Defining Member Functions Outside the Class	19
16.1	Why define functions outside?	20
17	Three Uses of the Scope Resolution Operator	20
17.1	Namespace qualification	20

17.2 Outside-class member definition	21
17.3 Accessing a hidden global name	21
18 Worked Program 1: Student Information	21
18.1 Problem statement	21
18.2 Sample execution	22
18.3 Discussion	23
19 Worked Program 2: Rectangle Analysis	23
20 Worked Program 3: Employee Salary	24
21 Reference Variables	25
21.1 Basic syntax	25
21.2 Conceptual memory view	26
21.3 Important rules	26
21.4 Reference is not a second independent variable	26
22 References as Function Parameters	26
22.1 Modification through a reference	27
22.2 Read-only reference	27
23 Worked Program 4: Swapping with References	27
23.1 Dry run	28
24 Worked Program 5: Bank Account	28
24.1 Design observations	30
25 Common Errors and Their Diagnosis	30
25.1 Missing semicolon after a class	30
25.2 Accessing a private member directly	30
25.3 Forgetting the class qualifier	30
25.4 Calling a member without an object	31
25.5 Uninitialised data	31

26 Concept Check Questions	31
27 Multiple-Choice Questions	32
28 Viva Questions with Short Answers	34
29 Programming Exercises	35
29.1 Level 1: Basic	35
29.2 Level 2: Intermediate	35
29.3 Level 3: Challenge	35
30 Debugging Exercises	36
30.1 Fragment 1	36
30.2 Fragment 2	36
30.3 Fragment 3	36
30.4 Fragment 4	37
31 Lecture Summary	37

1. Lecture Overview

1.1 Position of this lecture in Unit 1

Lecture 1 established why programming paradigms evolved and why object-oriented thinking is useful. Lecture 2 now moves from the conceptual foundation to the actual structure of a C++ program. It introduces the building blocks needed to write and understand simple object-oriented programs: program structure, tokens, classes, objects, access to members, the scope resolution operator and reference variables.

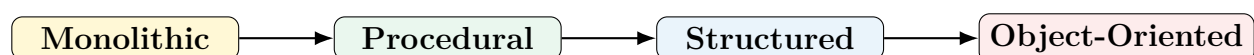
1.2 Learning outcomes

By the end of this lecture, a student should be able to:

1. identify the major parts of a C++ source file;
2. explain the role of the preprocessor, compiler, assembler and linker;
3. distinguish headers, namespaces, declarations, definitions and statements;
4. classify common C++ tokens;
5. define a class and create objects from it;
6. distinguish data members from member functions;
7. use the dot operator to access public members;
8. define member functions outside a class using the scope resolution operator;
9. explain the meaning of state, behaviour and identity of an object;
10. create and use a reference variable safely.

2. Recap of Lecture 1

The previous lecture discussed four broad stages in the evolution of programming styles:



Structured programming organises a program around functions. Object-oriented programming organises it around interacting objects that combine data and operations. C++ supports both styles, which makes it suitable for teaching the transition from procedural thinking to object-oriented design.

Remember

A language does not automatically make a program object-oriented. A C++ program can still be written entirely in procedural style. OOP depends on how the solution is designed.

3. Anatomy of a C++ Program

Consider the smallest useful C++ program:

```
1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "Hello, C++!\n";
6     return 0;
7 }
```

Listing 1: A minimal C++ program

This program contains several distinct elements:

1. a preprocessor directive: `#include <iostream>;`
2. the definition of the `main()` function;
3. a block enclosed within braces;
4. an output statement using `std::cout`;
5. a return statement.

3.1 Line-by-line explanation

3.1.1 The include directive

`#include <iostream>` asks the preprocessor to make declarations related to standard input and output available to the program. The name `iostream` refers to the input/output stream header.

3.1.2 The main function

Execution of a normal hosted C++ program begins with `main()`. The return type `int` indicates that the function returns an integer status code to the operating system.

3.1.3 The function body

The braces { and } define the body or block of the function. Statements inside the braces execute in sequence unless control-flow constructs alter that sequence.

3.1.4 The output statement

`std::cout` is the standard output stream. The insertion operator `«` sends data into that stream.

3.1.5 The return statement

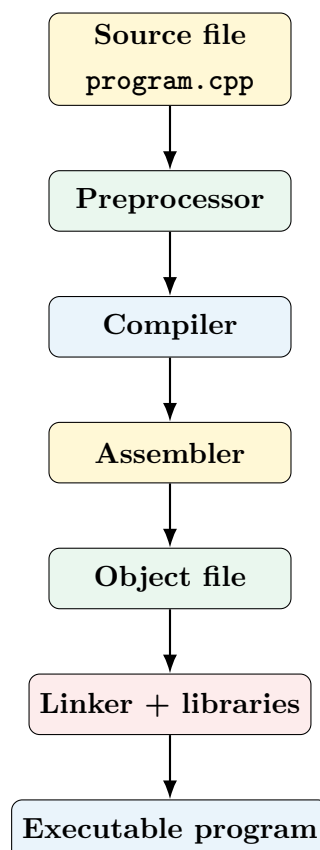
`return 0;` normally indicates successful completion. Reaching the closing brace of `main()` also implies a successful return in modern C++, but writing the statement explicitly is pedagogically useful.

Common Mistake

Do not write `void main()`. Standard C++ requires `main()` to return `int`.

4. From Source Code to Executable Program

A C++ program is not executed directly from the source file. It passes through a sequence of translation stages.



4.1 Preprocessing

The preprocessor handles directives beginning with `#`. Typical tasks include including headers, expanding macros and conditionally selecting code.

4.2 Compilation

The compiler checks syntax and semantics, performs translations and optimisations, and produces lower-level output.

4.3 Assembly

The assembler converts assembly instructions into machine-code object files.

4.4 Linking

The linker combines object files with required libraries and resolves external names. If a function is declared and used but no matching definition can be found, the linker may report an undefined reference.

Definition

A **compile-time error** is detected while translating the source code. A **link-time error** is detected while combining object files and libraries. A **run-time error** occurs while the executable is running.

Example

A missing semicolon usually produces a compile-time error. Calling a declared function whose definition is unavailable may produce a link-time error. Dividing by zero using run-time input may produce a run-time failure or undefined behaviour.

5. Preprocessor Directives and Header Files

5.1 Preprocessor directives

A preprocessor directive begins with the hash symbol `#`. It is processed before normal compilation.

```
1 #include <iostream>
2 #define MAX_STUDENTS 60
3
4 #ifdef DEBUG
```

```
5     #include <iomanip>
6 #endif
```

Listing 2: Common preprocessor directives

In modern C++, constants such as `MAX_STUDENTS` are generally better expressed using typed constants:

```
1 constexpr int maxStudents = 60;
```

5.2 Standard headers

Header	Typical purpose
<code><iostream></code>	Console input and output using streams.
<code><string></code>	The <code>std::string</code> class and related operations.
<code><iomanip></code>	Output formatting such as precision and field width.
<code><cmath></code>	Mathematical functions such as <code>sqrt</code> and <code>pow</code> .
<code><vector></code>	Dynamic sequence container <code>std::vector</code> .
<code><limits></code>	Properties and limits of numeric types.

5.3 Angle brackets and quotation marks

- `#include <name>` normally searches implementation-defined standard include locations.
- `#include "name"` normally begins with the project or current-file search path before searching standard locations.

Remember

A header is not the same as a library. A header primarily provides declarations needed for compilation; the corresponding implementation may be linked from a library.

6. Namespaces and the Scope Resolution Operator

Large programs may contain identical names created by different developers or libraries. Namespaces reduce such naming conflicts.

```
1 #include <iostream>
2 #include <string>
3
4 int main()
5 {
```

```
6     std::string name = "Aarav";
7     std::cout << name << '\n';
8 }
```

Listing 3: Qualified standard-library names

The expression `std::cout` means “the name `cout` that belongs to namespace `std`.” The symbol `::` is the scope resolution operator.

6.1 Using directive

The following directive makes names from `std` available without qualification:

```
1 using namespace std;
```

It is convenient in small classroom programs, but broad using directives should generally be avoided in header files and large projects because they can introduce ambiguity.

6.2 Selective using declaration

A more controlled alternative is:

```
1 using std::cout;
2 using std::string;
```

6.3 User-defined namespace

```
1 #include <iostream>
2
3 namespace College
4 {
5     int studentCount = 1200;
6 }
7
8 int main()
9 {
10     std::cout << College::studentCount << '\n';
11 }
```

Listing 4: A simple user-defined namespace

7. Statements, Blocks and Comments

7.1 Statements

A statement performs an action. Most simple statements end with a semicolon.

```
1 int marks = 85;  
2 marks = marks + 5;  
3 std::cout << marks;
```

7.2 Blocks

A block is a sequence of declarations and statements enclosed in braces. A block creates a scope.

```
1 {  
2     int temporary = 10;  
3     std::cout << temporary;  
4 }  
5 // temporary is no longer visible here
```

7.3 Comments

```
1 // Single-line comment  
2  
3 /* Multi-line  
4 comment */
```

Comments should explain intent, assumptions or non-obvious decisions. They should not repeat what clear code already says.

Common Mistake

The semicolon after a class definition is compulsory. Forgetting it is one of the most common errors made by beginners.

8. Tokens in C++

Definition

A **token** is a smallest meaningful lexical unit recognised by the compiler.

Common token categories are:

- keywords;
- identifiers;
- literals;
- operators;
- punctuators or separators.

Consider:

```
1 int total = marks + 5;
```

It contains:

- keyword: `int`;
- identifiers: `total`, `marks`;
- operators: `=`, `+`;
- integer literal: `5`;
- punctuator: `;`.

8.1 Keywords

Keywords have predefined meaning and cannot be used as ordinary identifiers. Examples include `class`, `public`, `private`, `int`, `return`, `if`, `while` and `const`.

8.2 Identifiers

Identifiers are programmer-defined names.

Rules include:

- the first character must be a letter or underscore;
- later characters may also include digits;
- spaces are not allowed;
- keywords cannot be used;
- identifiers are case-sensitive.

Valid	Invalid
<code>studentName</code>	<code>student name</code>
<code>marks2</code>	<code>2marks</code>

total_marks	total-marks
ClassRoom	class

8.3 Literals

A literal represents a fixed value written directly in the source code.

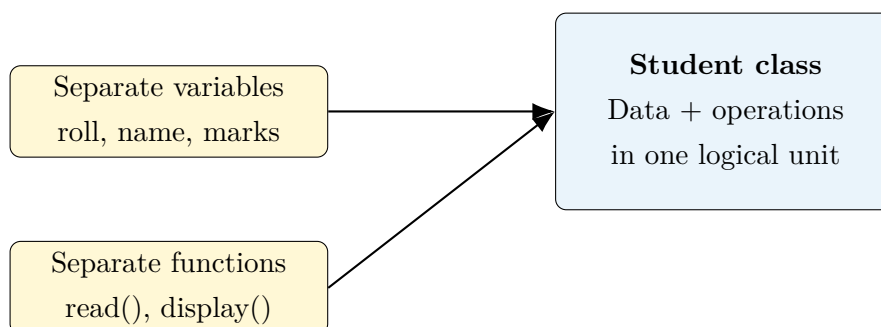
```

1 42          // integer literal
2 3.14        // floating-point literal
3 'A'         // character literal
4 "C++"       // string literal
5 true        // Boolean literal

```

9. From Structure to Object-Oriented Structure

A procedural program may store related values in separate variables and process them through independent functions. An object-oriented design groups related state and operations into a class.



Definition

A **class** is a programmer-defined type that describes the data and operations associated with a category of objects.

Definition

An **object** is an instance of a class. It possesses its own state and can use the behaviour defined by the class.

10. Class Syntax

```

1 class ClassName
2 {

```

```
3 private:
4     // data and functions accessible inside the class
5
6 public:
7     // interface accessible through objects
8 };
```

Listing 5: General syntax of a class

Important parts are:

- **class**: keyword introducing a class definition;
- **ClassName**: the name of the new type;
- **braces**: enclose the class body;
- **access specifiers**: control visibility;
- **semicolon**: terminates the class definition.

10.1 First class example

```
1 #include <iostream>
2 #include <string>
3
4 class Student
5 {
6 public:
7     int rollNumber;
8     std::string name;
9
10    void display()
11    {
12        std::cout << "Roll Number: " << rollNumber << '\n';
13        std::cout << "Name: " << name << '\n';
14    }
15 };
16
17 int main()
18 {
19     Student s1;
20     s1.rollNumber = 101;
21     s1.name = "Aditi";
22     s1.display();
23 }
```

Listing 6: A simple Student class

10.2 Dry run

1. The compiler learns a new type named **Student**.
2. **Student s1;** creates an object named **s1**.
3. The dot operator selects a member belonging to **s1**.
4. Values are assigned to the object's public data members.
5. **s1.display()** invokes the member function for that object.

11. Class versus Object

Class	Object
A user-defined type or blueprint.	A concrete instance of that type.
Describes permitted data and behaviour.	Stores actual state during execution.
Usually does not represent one particular entity.	Represents one identifiable entity.
Example: Student .	Examples: s1 , s2 .
Many objects may be created from one class.	Each object has its own non-static data members.

Example

A building plan is comparable to a class. Houses built from that plan are comparable to objects. The plan describes structure; each house has its own location, occupants and condition.

12. State, Behaviour and Identity

An object is commonly explained using three properties.

12.1 State

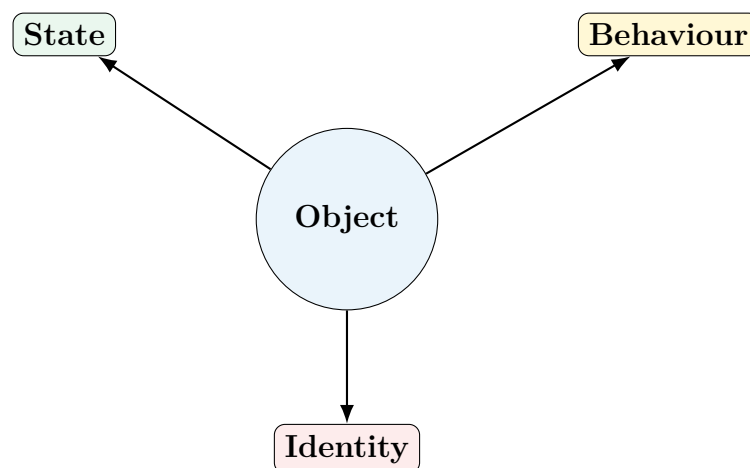
State is the collection of current values stored in the object's data members. For a bank account, state may include account number, holder name and balance.

12.2 Behaviour

Behaviour consists of the operations the object can perform, such as deposit, withdraw or display details.

12.3 Identity

Identity distinguishes one object from another even when their state happens to be equal.



13. Data Members and Member Functions

13.1 Data members

Variables declared inside a class are called data members. Each ordinary object receives its own copy of non-static data members.

13.2 Member functions

Functions declared inside a class are member functions. They operate in the context of an object and can directly access members of that class.

```
1 #include <iostream>
2
3 class Rectangle
```

```
4 {
5 public:
6     double length;
7     double width;
8
9     double area()
10    {
11        return length * width;
12    }
13 };
14
15 int main()
16 {
17     Rectangle room;
18     room.length = 6.5;
19     room.width = 4.0;
20
21     std::cout << "Area = " << room.area() << '\n';
22 }
```

Listing 7: Rectangle class with data and behaviour

Remember

A member function belongs logically to the class. The data on which it operates belongs to a particular object unless the member is declared static.

14. Creating Multiple Objects

A class may be used to create many independent objects.

```
1 #include <iostream>
2 #include <string>
3
4 class Book
5 {
6 public:
7     std::string title;
8     double price;
9
10    void display()
11    {
12        std::cout << title << " : Rs. " << price << '\n';
13    }
```

```
14 };
15
16 int main()
17 {
18     Book b1;
19     Book b2;
20
21     b1.title = "C++ Fundamentals";
22     b1.price = 499.0;
23
24     b2.title = "Data Structures";
25     b2.price = 599.0;
26
27     b1.display();
28     b2.display();
29 }
```

Listing 8: Multiple objects of one class

14.1 Conceptual memory view

Object b1

title = C++ Fundamentals
price = 499

Object b2

title = Data Structures
price = 599

The two objects use the same class definition, but each stores separate values.

15. Access Specifiers: A First Look

The main access specifiers are:

- `private;`
- `public;`
- `protected.`

For a class, members are private by default. Public members form the externally accessible interface.

```
1 #include <iostream>
2
3 class BankAccount
```

```
4 {
5 private:
6     double balance = 0.0;
7
8 public:
9     void deposit(double amount)
10    {
11        if (amount > 0)
12        {
13            balance += amount;
14        }
15    }
16
17    void showBalance()
18    {
19        std::cout << "Balance: " << balance << '\n';
20    }
21 };
22
23 int main()
24 {
25     BankAccount account;
26     account.deposit(1500.0);
27     account.showBalance();
28 }
```

Listing 9: Private data with public operations

Common Mistake

The statement `account.balance = -5000;` is invalid because `balance` is private. This restriction is useful: the class controls how its state changes.

16. Defining Member Functions Outside the Class

A member function may be declared inside a class and defined later. The scope resolution operator connects the definition to the class.

```
1 #include <iostream>
2
3 class Calculator
4 {
5 public:
6     int add(int first, int second);
```

```
7 };
8
9 int Calculator::add(int first, int second)
10 {
11     return first + second;
12 }
13
14 int main()
15 {
16     Calculator calc;
17     std::cout << calc.add(12, 8) << '\n';
18 }
```

Listing 10: Outside-class member-function definition

The definition:

```
int Calculator::add(int first, int second)
```

means that `add` belongs to the scope of `Calculator`.

16.1 Why define functions outside?

- the class declaration remains compact;
- interface and implementation can be separated;
- declarations may be placed in header files;
- definitions may be placed in source files;
- large projects become easier to navigate.

Common Mistake

Writing only `int add(...)` outside the class defines a different free function. The class name and `::` are required for the member-function definition.

17. Three Uses of the Scope Resolution Operator

17.1 Namespace qualification

```
1 std::cout << "Hello";
```

17.2 Outside-class member definition

```
1 void Student::display()
2 {
3     // ...
4 }
```

17.3 Accessing a hidden global name

```
1 #include <iostream>
2
3 int value = 100;
4
5 int main()
6 {
7     int value = 20;
8
9     std::cout << "Local: " << value << '\n';
10    std::cout << "Global: " << ::value << '\n';
11 }
```

Listing 11: Accessing a global variable hidden by a local variable

The leading `::value` refers to the global namespace.

18. Worked Program 1: Student Information

18.1 Problem statement

Create a class named **Student** that stores roll number, name and marks. Read values through a member function and display the record with the result status.

```
1 #include <iostream>
2 #include <string>
3
4 class Student
5 {
6 private:
7     int rollNumber{};
8     std::string name;
9     double marks{};
10 }
```

```
11 public:
12     void read();
13     void display();
14 };
15
16 void Student::read()
17 {
18     std::cout << "Enter roll number: ";
19     std::cin >> rollNumber;
20
21     std::cout << "Enter name: ";
22     std::getline(std::cin >> std::ws, name);
23
24     std::cout << "Enter marks: ";
25     std::cin >> marks;
26 }
27
28 void Student::display()
29 {
30     std::cout << "\nStudent Record\n";
31     std::cout << "Roll Number: " << rollNumber << '\n';
32     std::cout << "Name: " << name << '\n';
33     std::cout << "Marks: " << marks << '\n';
34     std::cout << "Result: " << (marks >= 40 ? "Pass" : "Fail") << '\n'
35         ;
36 }
37
38 int main()
39 {
40     Student student;
41     student.read();
42     student.display();
43 }
```

Listing 12: Student information program

18.2 Sample execution

Enter roll number: 17
Enter name: Kavya Sharma
Enter marks: 82.5

Student Record

Roll Number: 17

Name: Kavya Sharma

Marks: 82.5

Result: Pass

18.3 Discussion

- Data members are private, so outside code cannot alter them directly.
- Public functions provide controlled access.
- `std::ws` removes leading whitespace before `getline` reads the full name.
- Both member functions are defined outside the class.

19. Worked Program 2: Rectangle Analysis

```
1  #include <iostream>
2
3  class Rectangle
4  {
5  private:
6      double length{};
7      double width{};
8
9  public:
10     void setDimensions(double newLength, double newWidth)
11     {
12         if (newLength > 0 && newWidth > 0)
13         {
14             length = newLength;
15             width = newWidth;
16         }
17     }
18
19     double area()
20     {
21         return length * width;
22     }
23
24     double perimeter()
25     {
26         return 2 * (length + width);
```

```
27     }
28 };
29
30 int main()
31 {
32     Rectangle plot;
33     plot.setDimensions(12.5, 8.0);
34
35     std::cout << "Area: " << plot.area() << '\n';
36     std::cout << "Perimeter: " << plot.perimeter() << '\n';
37 }
```

Listing 13: Rectangle with area and perimeter

Teacher's Note

Ask students why the dimensions are not made public. Lead them to the idea that a public setter can reject invalid values, whereas unrestricted public data cannot enforce such rules.

20. Worked Program 3: Employee Salary

```
1  #include <iostream>
2  #include <string>
3
4  class Employee
5  {
6  private:
7      int id{};
8      std::string name;
9      double basicSalary{};
10
11 public:
12     void setData(int newId, const std::string& newName, double salary)
13     {
14         id = newId;
15         name = newName;
16         basicSalary = salary;
17     }
18
19     double grossSalary()
20     {
21         const double hra = 0.20 * basicSalary;
22         const double da = 0.10 * basicSalary;
```

```
23     return basicSalary + hra + da;
24 }
25
26 void display()
27 {
28     std::cout << "ID: " << id << '\n';
29     std::cout << "Name: " << name << '\n';
30     std::cout << "Gross Salary: " << grossSalary() << '\n';
31 }
32 };
33
34 int main()
35 {
36     Employee employee;
37     employee.setData(501, "Rohan Mehta", 40000.0);
38     employee.display();
39 }
```

Listing 14: Employee salary calculation

The parameter `const std::string& newName` is a constant reference. It avoids copying the string and prevents the function from modifying the caller's string through that parameter. References are introduced formally in the next section.

21. Reference Variables

Definition

A **reference variable** is an alias for an existing object. After binding, both names refer to the same object.

21.1 Basic syntax

```
1 #include <iostream>
2
3 int main()
4 {
5     int marks = 75;
6     int& alias = marks;
7
8     alias = 90;
9
10    std::cout << marks << '\n';    // 90
```

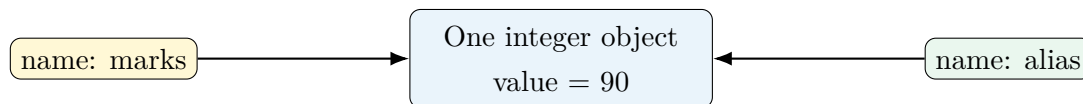
```

11     std::cout << alias << '\n';    // 90
12 }

```

Listing 15: Basic reference variable

21.2 Conceptual memory view



21.3 Important rules

- A reference must normally be initialised when declared.
- A non-const lvalue reference cannot bind directly to an unrelated temporary value.
- Once bound, a reference cannot be reseated to refer to another object.
- Operations through the reference affect the original object.

```

1  int number = 10;
2  int& ref = number;           // correct
3
4  // int& invalid;           // error: no initializer
5  // int& another = 25;      // error for a non-const lvalue reference

```

21.4 Reference is not a second independent variable

```

1  int first = 10;
2  int second = 20;
3  int& ref = first;
4
5  ref = second;

```

The final statement assigns the value of `second` to `first`. It does not make `ref` start referring to `second`.

22. References as Function Parameters

References become especially useful when passing arguments to functions.

22.1 Modification through a reference

```
1 #include <iostream>
2
3 void increment(int& value)
4 {
5     ++value;
6 }
7
8 int main()
9 {
10     int count = 5;
11     increment(count);
12     std::cout << count << '\n';
13 }
```

Listing 16: Increment using a reference parameter

22.2 Read-only reference

```
1 #include <iostream>
2 #include <string>
3
4 void printName(const std::string& name)
5 {
6     std::cout << name << '\n';
7 }
```

Listing 17: Constant reference parameter

A constant reference is useful for reading a potentially large object without making a copy.

Remember

Use a non-const reference when a function is intended to modify the caller's object. Use a const reference when the function should read the object without copying it.

23. Worked Program 4: Swapping with References

```
1 #include <iostream>
2
3 void swapValues(int& first, int& second)
4 {
5     int temporary = first;
```

```

6     first = second;
7     second = temporary;
8 }
9
10 int main()
11 {
12     int a = 10;
13     int b = 25;
14
15     std::cout << "Before: " << a << ' ' << b << '\n';
16     swapValues(a, b);
17     std::cout << "After: " << a << ' ' << b << '\n';
18 }

```

Listing 18: Swapping two integers using references

23.1 Dry run

Step	a/first	b/second	temporary
Before call	10	25	—
After temporary assignment	10	25	10
After <code>first = second</code>	25	25	10
After <code>second = temporary</code>	25	10	10

24. Worked Program 5: Bank Account

```

1 #include <iostream>
2 #include <string>
3
4 class BankAccount
5 {
6 private:
7     long accountNumber{};
8     std::string holderName;
9     double balance{};
10
11 public:
12     void open(long number, const std::string& holder, double
        openingBalance)
13     {
14         accountNumber = number;

```

```
15     holderName = holder;
16     balance = openingBalance >= 0 ? openingBalance : 0;
17 }
18
19 bool deposit(double amount)
20 {
21     if (amount <= 0)
22     {
23         return false;
24     }
25     balance += amount;
26     return true;
27 }
28
29 bool withdraw(double amount)
30 {
31     if (amount <= 0 || amount > balance)
32     {
33         return false;
34     }
35     balance -= amount;
36     return true;
37 }
38
39 void display()
40 {
41     std::cout << "Account: " << accountNumber << '\n';
42     std::cout << "Holder: " << holderName << '\n';
43     std::cout << "Balance: " << balance << '\n';
44 }
45 };
46
47 int main()
48 {
49     BankAccount account;
50     account.open(1001001, "Neha Verma", 5000.0);
51
52     account.deposit(1500.0);
53     if (!account.withdraw(8000.0))
54     {
55         std::cout << "Withdrawal rejected.\n";
56     }
57 }
```

```
58     account.display();  
59 }
```

Listing 19: A small object-oriented bank account program

24.1 Design observations

- The balance is protected from unrestricted direct modification.
- Each operation checks its own validity conditions.
- Boolean return values report whether an operation succeeded.
- The object maintains its own state across method calls.

25. Common Errors and Their Diagnosis

25.1 Missing semicolon after a class

```
1  class Test  
2  {  
3  public:  
4      int value;  
5  } // error: semicolon missing
```

25.2 Accessing a private member directly

```
1  class Test  
2  {  
3  private:  
4      int value;  
5  };  
6  
7  int main()  
8  {  
9      Test t;  
10     // t.value = 5; // access error  
11 }
```

25.3 Forgetting the class qualifier

```
1 class Test
2 {
3 public:
4     void show();
5 };
6
7 // Incorrect if this is intended to define Test::show
8 void show()
9 {
10 }
```

25.4 Calling a member without an object

```
1 class Test
2 {
3 public:
4     void show() {}
5 };
6
7 int main()
8 {
9     // show();           // not found as a free function
10    Test t;
11    t.show();           // correct
12 }
```

25.5 Uninitialised data

Fundamental-type data members may hold indeterminate values if not initialised. Prefer value initialisation or explicit initialisers.

```
1 class Counter
2 {
3 private:
4     int count{}; // initialised to zero
5 };
```

26. Concept Check Questions

1. Why is `main()` special in a C++ program?

2. What is the difference between preprocessing and compilation?
3. Why is `std::` written before `cout`?
4. What is the role of a semicolon after a class definition?
5. Distinguish a class from an object.
6. What is meant by object state?
7. What is meant by object behaviour?
8. Why can two objects of the same class store different values?
9. What does the dot operator do?
10. What does `Calculator::add` mean?
11. Why should data often be private?
12. What is a reference variable?
13. Can a reference be declared without an initializer?
14. Does assignment to a reference reseal the reference?
15. Why is `const std::string&` useful as a parameter?

27. Multiple-Choice Questions

1. Which function is the usual entry point of a C++ program?
 - (a) `start()`
 - (b) `main()`
 - (c) `run()`
 - (d) `init()`
2. Which operator qualifies a name with its scope?
 - (a) `.`
 - (b) `->`
 - (c) `::`
 - (d) `:`
3. Which statement creates an object of class `Student`?
 - (a) `class Student;`
 - (b) `Student s;`
 - (c) `object Student;`

(d) `new class Student;`

4. What terminates a class definition?

- (a) comma
- (b) colon
- (c) semicolon
- (d) full stop

5. Members of a C++ class are what by default?

- (a) public
- (b) private
- (c) protected
- (d) global

6. Which operator accesses a public member through an ordinary object?

- (a) dot
- (b) scope resolution
- (c) address-of
- (d) multiplication

7. A reference variable is best described as:

- (a) a copy
- (b) an alias
- (c) a class
- (d) a literal

8. Which declaration is valid?

- (a) `int& r;`
- (b) `int x; int& r=x;`
- (c) `ref int r=x;`
- (d) `int r&=x;`

9. Which stage combines object files and libraries?

- (a) preprocessor
- (b) linker
- (c) editor
- (d) loader only

10. Which is a valid identifier?

- (a) 2marks
- (b) total-marks
- (c) totalMarks
- (d) class

Answer key

1-(b), 2-(c), 3-(b), 4-(c), 5-(b), 6-(a), 7-(b), 8-(b), 9-(b), 10-(c).

28. Viva Questions with Short Answers

No.	Question	Suggested answer
1	What is a source file?	A text file containing program source code, commonly using the <code>.cpp</code> extension for C++.
2	What is a header?	A file that commonly provides declarations needed by source files.
3	What does <code>#include</code> do?	It asks the preprocessor to include the contents or declarations of a header.
4	Why does <code>main()</code> return <code>int</code> ?	It reports program termination status to the host environment.
5	What is a token?	A smallest meaningful lexical unit of source code.
6	What is an identifier?	A programmer-defined name for an entity.
7	Define class.	A user-defined type describing related data and operations.
8	Define object.	An instance of a class.
9	What is state?	Current values stored in an object's data members.
10	What is behaviour?	Operations offered by the object through member functions.
11	What is identity?	The distinction of one object from another.
12	What is a data member?	A variable declared inside a class.
13	What is a member function?	A function declared as part of a class.
14	What does the dot operator do?	It selects a member through an object.
15	What is <code>::</code> ?	The scope resolution operator.

16	Why are private members useful?	They restrict uncontrolled external access to object state.
17	Can one class create many objects?	Yes; each object normally has its own non-static state.
18	What is a reference?	An alias for an existing object.
19	Must a reference be initialised?	Normally yes, at declaration.
20	What is a const reference?	A reference through which the referred object cannot be modified.

29. Programming Exercises

29.1 Level 1: Basic

1. Create a class `Circle` with radius and a function that returns area.
2. Create a class `Product` with product code, name and price. Display one object.
3. Create two objects of a `Laptop` class and display their brand and price.
4. Write a program that demonstrates a reference alias for an integer.
5. Define a member function outside its class using `::`.

29.2 Level 2: Intermediate

1. Design a class `Temperature` that stores Celsius and returns Fahrenheit.
2. Design a class `Time` storing hour and minute. Reject invalid values through a public setter.
3. Create a class `Result` storing marks in three subjects and computing total, percentage and pass/fail status.
4. Write a function using reference parameters to return both quotient and remainder.
5. Create a class `InventoryItem` with controlled stock addition and removal.

29.3 Level 3: Challenge

1. Build a menu-driven bank account program using a class. Support deposit, withdrawal and balance enquiry.
2. Build a student record program that stores three objects in separate variables and prints the topper.
3. Design a class `ElectricityBill` that stores consumer details and computes a slab-based bill.

4. Separate a class declaration into a header file and definitions into a source file.
5. Compare a procedural student-record implementation with a class-based implementation and write a short design justification.

30. Debugging Exercises

Correct the errors in each fragment.

30.1 Fragment 1

```
1 class Box
2 {
3 public:
4     double length
5 }
```

30.2 Fragment 2

```
1 class Demo
2 {
3 private:
4     int value;
5 };
6
7 int main()
8 {
9     Demo d;
10    d.value = 10;
11 }
```

30.3 Fragment 3

```
1 class Printer
2 {
3 public:
4     void print();
5 };
6
7 void print()
```

```
8 {  
9     std::cout << "Printing";  
10 }
```

30.4 Fragment 4

```
1 int main()  
2 {  
3     int& reference;  
4     reference = 10;  
5 }
```

31. Lecture Summary

- A C++ source file contains directives, declarations, definitions and statements.
- The source code is preprocessed, compiled, assembled and linked before execution.
- Headers provide declarations required by source files.
- Namespaces organise names and reduce collisions.
- The scope resolution operator qualifies names with a namespace, class or global scope.
- Tokens include keywords, identifiers, literals, operators and punctuators.
- A class defines a user-defined type; an object is an instance of that type.
- Objects are described by state, behaviour and identity.
- Data members store state; member functions define behaviour.
- Private data supports controlled access and valid object state.
- Member functions may be declared inside a class and defined outside using `ClassName::functionName`.
- A reference is an alias for an existing object and must normally be initialised immediately.